Day 1

Python Fundamentals

+ build the Catalog Foundation

6 hours · 3 modules · 3 labs · one repo for the week

What we build today

By 6 PM, a **local FastAPI server** running on your laptop, serving a `Product` catalog you wrote from scratch.

- Module 1 → Python core → Lab 1: `Product` foundation
- Module 2 → Data structures, files, modules → Lab 2: Persistent catalog
- Module 3 → OOP, decorators, FastAPI → Lab 3: Local API server

Same repo carries through Day 2, 3, 4. No throwaway demos.

The week, in one picture

```
Day 1   Product class + FastAPI server
Day 2   + Pydantic + APIClient + CSV bulk-import
Day 3   + pytest suite + mocks + CI green
Day 4   + LLM-powered CatalogAgent + agent tests
```

Every day extends yesterday's `product-catalog/` repo.

A catch-up baseline (`day-N-start/`) is provided each morning.

Module 1

Python Core

~40 min · then 80 min lab

Why we start with the basics

Even seasoned engineers skip the *shapes* of Python that everything else leans on:

- **Functions** are values — pass them around
- **Exceptions** are control flow — design them
- `*args` / `**kwargs` are how decorators work (Module 3)
- **Type hints** are how FastAPI and pytest read your intent

Get these right today and Days 2-4 stop feeling magical.

Variables, types, operators (quick)

```python
name: str = "Mechanical Keyboard"
price: float = 5499.0
in_stock: bool = True
tags: list[str] = ["keyboard", "mech"]

# str / int / float / bool / list / dict / tuple / set / None
# operators: + - * / // % ** == != < > and or not is in
```

Python is dynamically typed — annotations are *for readers and tools*,
not enforced at runtime. (Unless something like Pydantic enforces them — Day 2.)

Control flow & truthiness

```
def categorize(price: float) → str:
    if price < 500:
        return "budget"
    elif price < 5000:
        return "mid"
    else:
        return "premium"

for tag in product.tags:
    if "mech" in tag.lower():
        print("mechanical!")
        break
```

Falsy: `0`, `0.0`, `""`, `[]`, `{}`, `None`, `False`.

Everything else is truthy. Don't write `if x == None:` — write `if x is None:`.

Functions — the real shape

```
def add_product(
    catalog,
    name: str,
    price: float,
    *,                          # everything after is keyword-only
    category: str = "Misc",
    tags: list[str] | None = None,
) → int:
    tags = tags or []
    ...
    return product_id
```

- `*args` collects positional, `**kwargs` collects keyword
- `*` alone forces keyword-only — great for clarity at call sites
- A function's return type is the *contract* readers trust

Exception handling

```python
class CatalogError(Exception):
    """Raised when a catalog operation fails."""

try:
    catalog.add(product)
except CatalogError as exc:
    logger.warning("could not add: %s", exc)
except Exception:
    logger.exception("unexpected")  # logs traceback
    raise
```

Three rules:

1. **Define your own exception types** for your domain (`CatalogError`).
2. **Catch narrowly**, log, decide whether to re-raise.
3. **Never `except: pass`** — it eats real bugs.

Mental model for today's lab

```
Product        ← dataclass: id, name, category, price, in_stock, tags
ProductCatalog ← dict[int, Product] inside
                   .add  .get  .delete  .list_all  .__len__
CatalogError   ← your own exception
```

That's it. ~80 lines of Python.

🧪 Lab 1 — The `Product` Foundation

80 min · open `labs/lab-01-product-foundation/README.md` · scaffolds in `starter/`

You'll build:

- `catalog/models.py` — `Product` + `ProductCatalog`
- `catalog/cli.py` — `list`, `add` subcommands

End state: `python -m catalog.cli list` prints 5 seeded products.

Module 2

Data Structures, Files & Modules

~40 min · then 80 min lab

The four containers (and when each fits)

	use it for	gotcha
`list`	ordered, mutable, dupes OK	O(n) lookup
`tuple`	fixed-shape record, hashable	immutable — feature, not bug
`set`	unique membership	unordered
`dict`	**lookup by key**, JSON model	keys must be hashable

`dict` is the model for JSON. Master dict and Day 2 gets easy.

Dict — go deep

```
catalog = {1: product_a, 2: product_b}

# safe lookup
catalog.get(99, default_product)

# merge (3.9+)
combined = catalog | other_catalog

# iterate
for pid, product in catalog.items():
    ...

# common bug: mutating while iterating
for pid in list(catalog):           # copy keys first
    if catalog[pid].price < 100:
        del catalog[pid]
```

Comprehensions — the Python idiom

```python
# list comprehension
hits = [p for p in products if "mech" in p.name.lower()]

# dict comprehension
by_id = {p.id: p for p in products}

# nested + filter
expensive = {cat: [p for p in items if p.price > 1000]
             for cat, items in groups.items()}
```

Comprehensions express *what*, not *how*. They replace 4-line for-loops.

Don't nest more than 2 levels deep — at that point write a normal loop.

File handling

```python
# JSON
import json
payload = [p.to_dict() for p in catalog.list_all()]
Path("catalog.json").write_text(json.dumps(payload, indent=2))

# CSV (always pass newline="")
import csv
with open("catalog.csv", "w", newline="") as fh:
    writer = csv.DictWriter(fh, fieldnames=["id", "name", "price"])
    writer.writeheader()
    for p in catalog.list_all():
        writer.writerow(p.to_dict())
```

CSV is text-with-rules — quotes, escapes, encodings will bite. Don't roll your own.

Modules & virtual environments

```
product-catalog/
├── pyproject.toml
└── catalog/
    ├── __init__.py
    ├── models.py
    ├── storage.py
    └── cli.py
```

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -e ".[dev]"            # editable install + dev extras
python -m catalog.cli list
```

`uv` is the faster modern alternative — `uv venv` + `uv pip install`.

Logging (not `print`)

```
import logging
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO,
                    format="%(levelname)s %(name)s: %(message)s")

logger.info("added product id=%s", product.id)
logger.warning("price unusually high: %s", price)
logger.exception("save failed")    # includes traceback
```

- Loggers are *named* — pass `__name__` and you can tune per-module.
- Pass values as args (`%s`), not f-strings — lazy formatting + structured logs.
- `print()` belongs in scripts; libraries should *only* log.

🧪 Lab 2 — Persistent Catalog

80 min · open `labs/lab-02-persistent-catalog/README.md` · scaffolds in `starter/`

You'll build:

- `catalog/storage.py` — CSV + JSON load/save
- Comprehension methods: `search_by_name`, `filter_by_price`, `group_by_category`
- `search` / `save` / `load` CLI subcommands

End state: catalog survives process restart.

Module 3

OOP, Decorators, Type Hints, FastAPI

~40 min · then 80 min lab

Classes — the minimum that pays back

```
class ProductCatalog:
    def __init__(self, products: list[Product] | None = None) → None:
        self._items: dict[int, Product] = {}
        for p in products or []:
            self.add(p)

    def add(self, product: Product) → Product:
        if product.id in self._items:
            raise CatalogError( ... )
        self._items[product.id] = product
        return product
```

- `__init__` builds state, methods change it
- `self._items` (single underscore) = "private by convention"
- Don't subclass until you've written it twice — composition first

`dataclass` — the bridge to Pydantic

```python
from dataclasses import dataclass, field

@dataclass
class Product:
    id: int
    name: str
    category: str
    price: float
    in_stock: bool = True
    tags: list[str] = field(default_factory=list)
```

Generated for free: `__init__`, `__repr__`, `__eq__`.

Day 2 swaps `@dataclass` → Pydantic `BaseModel` and gains runtime validation.

The fields stay identical.

Type hints — why they matter

```
def filter_by_price(products: list[Product], max_price: float) → list[Product]:
    return [p for p in products if p.price ⩽ max_price]
```

Hints are not enforced by Python, but:

- **Editors** autocomplete from them
- **mypy / pyright** catch bugs before runtime
- **FastAPI** uses them to generate OpenAPI schemas
- **Pydantic** uses them to *validate*

Get used to writing them — Days 2-4 lean on them hard.

Decorators — read one before writing one

```
def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        logger.info("call %s", func.__name__)
        result = func(*args, **kwargs)
        logger.info("return %s → %r", func.__name__, result)
        return result
    return wrapper

@log_calls
def add_product(catalog, product):
    ...
```

A decorator is a function that **takes a function and returns a function**.

`@log_calls` is the same as `add_product = log_calls(add_product)`.

Parametrized decorators (`@retry(times=3)`)

```
def retry(times=3, delay=0.1, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as exc:
                    if attempt < times:
                        time.sleep(delay)
            raise exc
        return wrapper
    return decorator
```

Three nested functions. Once you see it once, every later decorator is the same shape.

FastAPI is decorators in action

```python
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Product Catalog")

@app.get("/products")
def list_products() → list[dict]:
    return [p.to_dict() for p in catalog.list_all()]

@app.post("/products", status_code=201)
def create_product(payload: dict) → dict:
    ...
```

- `@app.get("/products")` *registers* the function as a route
- Type hints determine request parsing + response shape
- `/docs` Swagger UI is auto-generated

This is the *exact* pattern Day 4's `@tool` will reuse for the agent

What you'll run at the end of Lab 3

```
uvicorn catalog.server:app --reload

# Another terminal
curl http://localhost:8000/health
# {"status":"ok","count":5}

curl http://localhost:8000/products/2
# {"id":2,"name":"Mechanical Keyboard", ...}
```

Then visit **http://localhost:8000/docs** — that's your API, documented for free.

🧪 Lab 3 — Local API Server

80 min · open `labs/lab-03-local-api-server/README.md` · scaffolds in `starter/`

You'll build:

- `catalog/decorators.py` — `@retry` and `@log_calls`
- `catalog/server.py` — FastAPI app with full CRUD
- A running server on `localhost:8000` with `/docs`

End of Day 1 → your repo matches `checkpoints/day-2-start/`.

End of Day 1 ✅

Tomorrow: typed payloads (Pydantic), `requests`, and a Python client that drives this server end-to-end.

Before you leave: commit your work, push to your branch, take a screenshot of `/docs`.